

# MCP Gateways: A Best-Practices Guide for Securing Agentic AI

What an MCP gateway is, why it is now the primary control point for agentic AI, and where Check Point fits — with AWS Bedrock AgentCore Gateway as a worked example.

## 01

### Executive Summary

As enterprises connect AI agents to real tools using the Model Context Protocol (MCP), the **MCP gateway** has become the single most important control point for security, governance, and scale. This guide makes vendor-neutral gateway best practices the backbone, treats AWS Bedrock AgentCore Gateway as one concrete worked example, and positions Check Point in its two distinct roles.

**2**

standard MCP transports: stdio &amp; Streamable HTTP

**18**

Check Point open-source MCP server packages (@chkp)

**54**

read-only tools in the Quantum Management MCP (verified hands-on)

**1**

officially announced Check Point + AgentCore integration

Two things to get right, and this guide is built around them:

## OFFICIAL INTEGRATION

The **only announced** Check Point + AWS Bedrock AgentCore integration (17 June 2026) is **Check Point AI Security acting as a guardrail / policy signal at the gateway** — deterministic allow/deny enforcement *outside* the agent's reasoning loop. It is *not* "register a Check Point management MCP server as a gateway tool."

## REFERENCE ARCHITECTURE — LABEL IT AS SUCH

Running a **Check Point MCP server as an AgentCore Gateway target** is a pattern this guide assembles from public AWS + Check Point docs. Every step is individually verified against primary docs, but there is **no joint AWS + Check Point reference architecture**. Present it as a synthesized design, never as a shipped joint solution.

02

# MCP Fundamentals Worth Understanding

MCP is an open, JSON-RPC 2.0-based protocol that standardizes how AI applications connect to external context and tools — think **"USB-C for AI tools."** Five concepts to be fluent in:

- **Client–host–server architecture.** The *host* (the AI application) creates and manages *client* instances, controls permissions and consent, and coordinates the LLM. Each *client* holds an isolated, stateful session with exactly one *server* (1:1). The 1:1 is directional — one remote server can serve many clients.
- **Stateful, capability-negotiated sessions.** Client and server declare supported features in an initialization handshake. (Evolving: a subset can run stateless over Streamable HTTP, and the upcoming `2026-07-28` revision introduces a stateless protocol core.)
- **Three server primitives:** *tools* (model-controlled actions, discovered via `tools/list` and invoked via `tools/call`), *resources* (application-controlled data via URIs), and *prompts* (user-controlled templates). Clients may offer sampling, roots, and elicitation.
- **Two standard transports:** `stdio` (local subprocess over stdin/stdout) and `Streamable HTTP` (remote; HTTP POST/GET with optional SSE, session via the `Mcp-Session-Id` header). Streamable HTTP replaced the deprecated 2024-11-05 HTTP+SSE transport.
- **Authorization is OAuth 2.1-based, OPTIONAL, and HTTP-only.** `stdio` servers read credentials from the environment. On HTTP, the MCP server is an OAuth 2.1 resource server; the spec mandates PKCE, RFC 9728 Protected Resource Metadata for discovery, and RFC 8707 resource indicators so tokens are **audience-bound** to a specific server.

## VERIFIED HANDS-ON

Driving a live handshake against two real Check Point servers negotiated protocol `2025-06-18` cleanly: `initialize` → `notifications/initialized` → `tools/list`. The isolation model (a server cannot read the full conversation or see into other servers) is exactly what makes a *gateway* the natural place to concentrate policy.

## The MCP Gateway Pattern (The Backbone)

An **MCP gateway** is an infrastructure layer that sits in front of one or more MCP servers to provide a single, secure entry point for AI clients. Its jobs:

- **Aggregate** many servers behind one endpoint (a unified tool catalog).
- **Centralize authentication / authorization** across all backends.
- **Enforce policy & tool allowlists** — what may be called, by whom.
- **Broker credentials server-side** so agents never see secrets.
- **Provide observability / audit** — one place for logs and metrics.
- **Route & version** — session affinity, backend lifecycle.

### Why gateways exist — the configuration explosion

Docker frames it precisely: a gateway turns an  $X \text{ applications} \times Y \text{ servers}$  problem into just  $Y$ . Each client connects once to the gateway, which manages every backend behind a unified interface — collapsing  $N\text{-clients} \times M\text{-servers}$  sprawl into one connection per client.

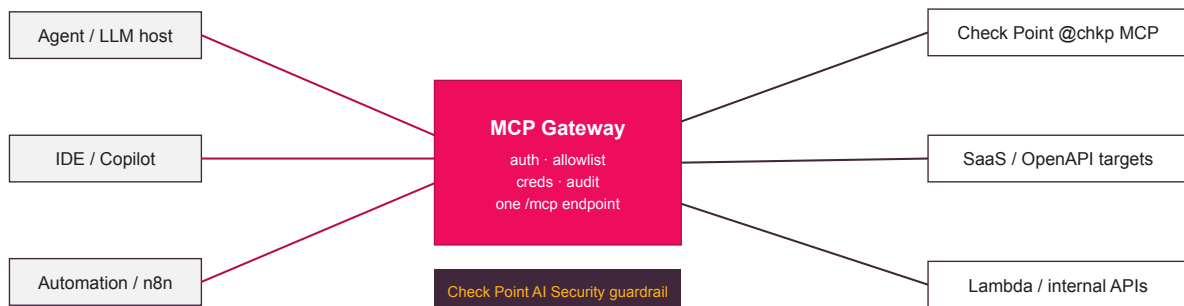


Figure 1 — The gateway is the choke point: agents connect once; the gateway fronts many MCP servers, brokers credentials, and is where a runtime guardrail (e.g. Check Point AI Security) can enforce allow/deny outside the agent's reasoning loop.

### Pattern categories & ecosystem

A commonly-cited (community, *not* spec-defined) taxonomy has five categories: transport bridging, server aggregation, security gateway, cloud-native gateway, and edge-based patterns. Present it as convention, not standard.

| Category      | Representative implementations                                                                                      |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Open-source   | mcp-proxy, supergateway, IBM ContextForge (mcp-context-forge), MetaMCP, Docker MCP Gateway, Lasso, Envoy AI Gateway |
| Cloud-managed | <b>AWS Bedrock AgentCore Gateway</b> , Kong, Azure API Management, WSO2, Gravitee APIM                              |

**TRADE-OFF — SAY THIS OUT LOUD**

A gateway **concentrates trust and risk into one control plane**. That is a high-value attack surface and a potential single point of failure — it must itself be hardened, monitored, and made highly available.

04

# MCP Gateway Best-Practices Checklist

The vendor-neutral controls a gateway (or a well-run MCP deployment) should implement. Items marked **SPEC** are normative MUST / MUST-NOT requirements of the MCP authorization and security specs; the rest are industry consensus.

## Identity & tokens

- **SPEC**. Enforce OAuth 2.1 + PKCE (S256); use RFC 9728 for AS discovery and RFC 8707 resource indicators so tokens are **audience-bound**. Validate audience server-side even if the authorization server ignores the resource parameter — that check is the real enforcement point.
- **SPEC**. **Never pass client tokens through to upstream APIs**. Token passthrough breaks audit, bypasses rate limits, and opens a confused-deputy / exfiltration path. Obtain a separate downstream token.
- Issue short-lived, scoped tokens; rotate refresh tokens for public clients. Practice **scope minimization** — start minimal and elevate progressively; avoid wildcard/omnibus scopes.
- Never use MCP sessions as an authentication mechanism; use non-deterministic session IDs bound to user identity.

## Tools & trust

- **Pin and hash tool definitions** (name + description + schema); alert / re-prompt on any change to defeat "rug-pulls." Version-lock MCP server packages.
- Maintain a **tool/server allowlist**; inspect full descriptions, parameter names, and return schemas before approval (scan for hidden or AI-only instructions). Use strict JSON Schema with `additionalProperties:false`.
- Require **human-in-the-loop** for destructive / financial / data-sharing / cross-server calls, showing full parameters — never summaries. Never auto-approve multi-server calls.
- Treat every tool *result* as untrusted input; scan/sanitize before returning it to the model. Treat third-party tool annotations as untrusted.

## Network & operations

- Broker credentials server-side and inject at execution time; **decouple inbound (agent→gateway) from outbound (gateway→backend) auth.**
- Apply egress / SSRF defenses: block private/reserved ranges and cloud metadata ( `169.254.169.254` ), require HTTPS, guard against DNS-rebinding, route server-side fetches through an egress proxy.
- Validate `Origin` headers; bind local servers to `127.0.0.1` (not `0.0.0.0` ); require auth on all HTTP connections.
- Centralize observability: log every invocation with parameters, user, and timestamp to a SIEM (redacting secrets/PII). Isolate sensitive servers on separate networks; run backends least-privileged.
- Vet the supply chain: install only from verified sources, review source and tool definitions, verify checksums/signatures, watch for typosquatting.

05

# The MCP Threat Model — What To Defend Against

MCP inherits classic AppSec / OAuth risks *plus* a new agent-specific class where the LLM treats tool metadata and results as trusted instructions. Distinguish spec-codified threats from research taxonomy.

### SPEC-CODIFIED (MCP SECURITY BEST PRACTICES + AUTHORIZATION)

| Threat                  | Essence & mitigation                                                                                                                                                                                                                                       |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Token passthrough       | MUST NOT accept/forward tokens not issued for the server; MUST validate audience.                                                                                                                                                                          |
| Confused deputy         | A proxy using a static client_id with a third-party AS can leak auth codes via an existing consent cookie. Mitigate: per-user approved-client registry checked before forwarding; exact redirect_uri matching; single-use state; hardened consent cookies. |
| SSRF                    | Malicious internal URLs in OAuth discovery fields. Block private ranges + metadata, HTTPS-only, DNS-rebinding guards, egress proxy.                                                                                                                        |
| Session hijacking       | Verify every inbound request; don't use sessions for auth; non-deterministic IDs bound to user identity.                                                                                                                                                   |
| Local server compromise | Show the exact untruncated install command, require approval, sandbox, flag dangerous patterns ( <code>sudo</code> , <code>rm -rf</code> ).                                                                                                                |

### RESEARCH / COMMUNITY TAXONOMY (DO NOT PRESENT AS SPEC TERMS)

- **Tool poisoning** — malicious instructions hidden in tool descriptions/schemas, invisible to users but read by the model.  
Invariant Labs' April 2025 PoC exfiltrated `~/ssh/id_rsa` via a benign-looking "add" tool. The highest-severity MCP-

specific class.

- **Rug-pull / mutable definitions** — a server silently changes a tool after approval, because most clients bind trust to the tool *name*, not its content. One-time vetting is insufficient.
- **Tool shadowing / name collision** — a malicious server's description manipulates the agent's use of a *different*, trusted server.

### PROVEN IN THE WILD

The **postmark-mcp** npm package (v1.0.16, Sept 2025) silently BCC'd every outgoing email to an attacker domain after ~15 clean, reputation-building releases — the first confirmed real-world malicious MCP server (~1,500 weekly downloads). **Lesson: change-detection and re-verification over time, not just install-time vetting.**

06

## Worked Example: AWS Bedrock AgentCore Gateway

AgentCore Gateway is AWS's **fully-managed AI gateway** — one of five AgentCore components (Runtime, Gateway, Identity, Memory, Observability). AgentCore reached **GA on 13 October 2025** in nine regions (adding VPC, PrivateLink, CloudFormation, tagging), after a July 2025 preview. It turns APIs, Lambda functions, and existing services into MCP-compatible tools.

### Targets & aggregation

Ingests four target types — **Lambda, OpenAPI specs, Smithy models, and existing/remote MCP servers** — plus 1-click connectors (Salesforce, Slack, Jira, Asana, Zendesk). In aggregation mode all targets combine into one virtual MCP server; clients see a single consolidated `tools/list`, with tools and prompts namespaced as `targetName__toolName` (three underscores; resources are not prefixed). Supported MCP protocol versions: `2025-06-18`, `2025-03-26`, `2025-11-25`.

### Client access & auth

- **Client access:** a standard MCP Streamable-HTTP client (MCP SDK, Strands, LangGraph, CrewAI) connects to the gateway's `/mcp` endpoint with an `Authorization: Bearer <JWT>` header. Get the hostname from the `CreateGateway` API response — do not hardcode.
- **Inbound auth:** OAuth/JWT authorizer (Cognito, Okta, Auth0; 2LO/3LO), IAM SigV4, or no-auth (dev/test only).
- **Outbound auth:** IAM execution role for Lambda/Smithy; AgentCore **Identity credential providers** (API key or OAuth 2LO/3LO) for OpenAPI/MCP-server targets; or a `GATEWAY_IAM_ROLE` **SigV4 provider** (`service=bedrock-agentcore`) when the MCP-server target is itself SigV4-verifying — e.g. an AgentCore Runtime (the path used in the field-tested build). Note: SigV4 outbound only works behind a SigV4-verifying service (Runtime/Gateway, API Gateway, Lambda Function URLs) — not a plain ALB/EC2.

## Semantic tool search & listing modes

- Built-in tool `x_amz_bedrock_agentcore_search` builds a vector index over tool metadata so agents find tools by natural-language query. **Critical: it can only be enabled at gateway-creation time.**
- **DEFAULT** listing (cached — call `SynchronizeGatewayTargets` after tool changes; async, can take minutes) vs **DYNAMIC** (live, caller-identity-aware, but not interoperable with semantic search or outbound 3LO).

## Post-GA additions

First-class prompts and resources, dynamic listing, streaming/session management (default 3600s; configurable 900s–28800s), elicitation, and **OAuth 2.0 on-behalf-of (OBO) token exchange** preserving user identity across every hop. Server-side tool execution via the gateway is also supported.

### OBSERVABILITY — DO NOT OVER-CLAIM

AgentCore emits OpenTelemetry-compatible telemetry with CloudWatch dashboards, plus CloudTrail (management events by default; data events only when explicitly enabled via an S3-backed trail). Per AWS's authoritative metrics reference, the Gateway emits **only**: `TargetType` (usage); and `Invocations, Throttles, SystemErrors, UserErrors, Latency, Duration, TargetExecutionTime` (the last three with avg/min/max/p50/p90/p99). Names like `IngressAuthType`, `RequestsPerSession`, `Sessions` appeared only in launch-blog copy — **do not cite them as Gateway metrics**. Datadog and Langfuse export are documented; Dynatrace is a plausible OTEL inference, not a confirmed integration.

07

## Where Check Point Fits — Two Complementary Roles

A customer must not conflate these. Check Point both *publishes MCP servers* (tools an agent can call) and *sells the AI-security enforcement layer* that polices agentic/MCP traffic.

## Role 1 — Check Point as MCP tools

The official open-source monorepo `github.com/CheckPointSW/mcp-servers` ships **18 MIT-licensed** `@chkp/` packages (Quantum Management, Management Logs, Threat Prevention, HTTPS Inspection, Harmony SASE, Reputation, GW-CLI, Threat Emulation, Gaia, Documentation, Spark, CPInfo, Argos ERM, Policy Insights, CloudGuard WAF, CPView History, Workforce AI, and more). They let an agent query your estate in natural language.

- **Transport:** default `stdio` via `npx @chkp/<pkg>`. An optional HTTP mode exists but ships with no built-in auth and no TLS (see gotcha below).
- **Auth (env vars, never shown to the model):** Smart-1 Cloud ( `API_KEY` + `S1C_URL` ); on-prem ( `MANAGEMENT_HOST` / `PORT` + `API_KEY` or `USERNAME` / `PASSWORD` ); Harmony SASE ( `API_KEY` + `MANAGEMENT_HOST` + `ORIGIN` ).
- **Read-oriented — but verify per package.** Quantum Management & Harmony SASE self-describe as read-only; several packages say "manage." Scope the API key to least privilege regardless.

### VERIFIED HANDS-ON (13 JUL 2026)

`@chkp/quantum-management-mcp` (v1.4.x) exposed **54 tools, every one read-only** ( `show_*` , `where_used` , `simulate_packet` , `find_zero_hits_rules` , plus a `management__init` login tool). No create/set/delete. Tool *registration* is static, so `tools/list` works even before the management API is reachable.

## Role 2 — Check Point as the enforcement layer

Check Point's "AI Defense Plane" spans Workspace AI security, AI application security, and **AI agent runtime security** (governing tool calls, blocking unsafe autonomy). Verified specifics:

- **CloudGuard WAF + Lakera dual-layer ML** — a supervised layer (Prompt Injection Prevention, Data Leakage Prevention, Content Control, Usage Control) plus an unsupervised refinement layer, 100+ languages, covering GenAI apps, APIs & agents.
- **Quantum Firewall R82.10 natively detects & monitors MCP traffic** — a shipped capability (Dec 2025 release), including auto-detection of unauthorized GenAI tools and MCP servers.
- **Lakera:** Check Point announced an agreement to acquire Lakera (Sept 2025). *Gandalf* — Lakera's adversarial red-team game — *feeds* the guardrails; the product is **Lakera Guard / AI Guardrails**. Do not equate Gandalf with the guardrail engine.

### THE CENTRAL TECHNICAL GOTCHA

Check Point MCP servers are `stdio` / `npx` by default, and their optional HTTP mode ( `MCP_TRANSPORT_TYPE=http` , port 3000, `/mcp` + `/health` ) ships with **no authentication and no TLS** — the README says to run it only behind an authenticated, TLS-terminating reverse proxy. There is no hosted/managed remote Check Point endpoint; servers run in the customer's environment. **Never expose a raw Check Point MCP HTTP port.**



### TLS POLICY FINDING — ON-PREM CLIENT

Separately, the on-prem client ( `@chkp/quantum-infra` ) hardcodes `new https.Agent({ rejectUnauthorized: false })` in `login()` and `callApi()` — it **disables TLS certificate verification** on the connection to `MANAGEMENT_HOST`. A self-signed SMS cert therefore won't block it, but this **conflicts with the "never disable TLS verification" rule** (and `NODE_EXTRA_CA_CERTS` is inert while it's forced). **Smart-1 Cloud** ( `S1C_URL` ) keeps verification *on* — prefer it, or fork/patch `quantum-infra` to trust the SMS CA. Worth flagging to the `@chkp` product team.

08

## Check Point + AWS AgentCore: Official vs. Reference Architecture

This is the section to get exactly right. There are two separate integration stories, and only one is officially announced.

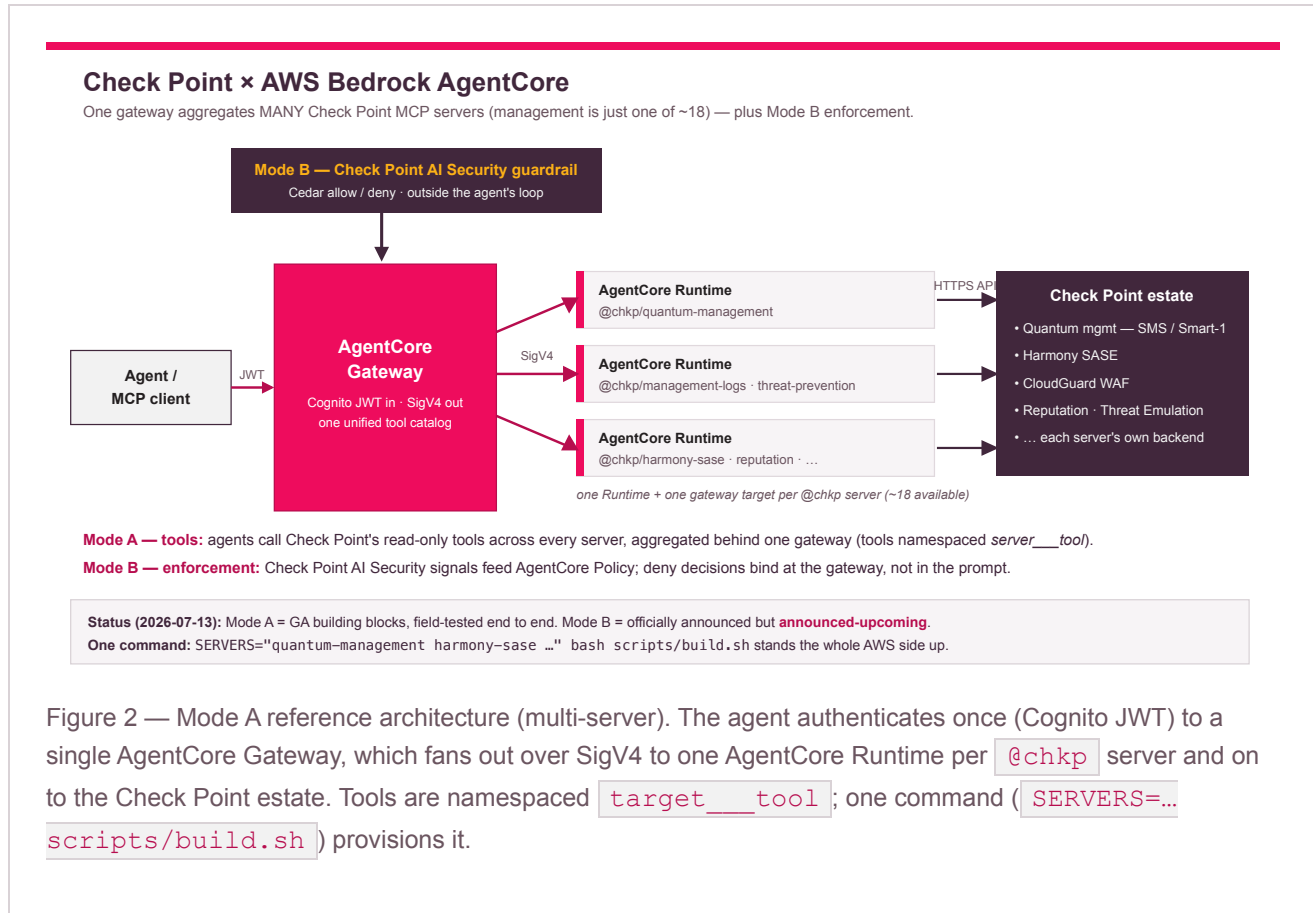
### OFFICIAL — ANNOUNCED 17 JUNE 2026

**Check Point AI Security as an AgentCore Gateway guardrail.** Check Point supplies detection signals — prompt-injection detection, sensitive-data/DLP indicators, content inspection, behavioral threat detection — that feed AgentCore policy **at the gateway, outside the agent's reasoning loop**, enabling deterministic allow/deny. Check Point framed it as moving beyond *defining what agents may do* toward *ensuring they do the right thing*. This is a security-guardrail pattern, not Check Point management servers registered as tools.

### SYNTHESIZED REFERENCE ARCHITECTURE — NO OFFICIAL JOINT BLUEPRINT

**Check Point MCP server as an AgentCore Gateway target.** Assembled from generic AWS MCP-target docs + Check Point's public repo. Mechanics below are each verified against AWS docs — but never jointly with Check Point. Label it clearly when presenting.

## How the reference architecture works



- 1. Register the target.** AgentCore Gateway supports an **MCP server target** (`McpTargetConfiguration.McpServer`) that registers an existing MCP server at an HTTPS endpoint, aggregates it, and namespaces its tools as `targetName__tool` (three underscores). The server must advertise tool capabilities; prompts/resources are optional.
- 2. Host it correctly (this is where most attempts break).** AgentCore requires a **publicly-trusted-TLS HTTPS endpoint** and does not trust private-CA certs — so host the Check Point server on **AgentCore Runtime** as an **ARM64** container listening on `0.0.0.0:8000/mcp` (built in-cloud with CodeBuild; no local Docker needed). The `@chkp` server **manages its own MCP sessions** (it issues its own `Mcp-Session-Id`), so drive it with clients that complete the `initialize` handshake and reuse that session id — AgentCore exposes no stateless/stateful toggle to set. Runtime supplies the TLS + inbound-auth envelope the Check Point HTTP transport deliberately lacks.
- 3. Inbound auth (caller → gateway):** a Cognito (or Okta/Auth0) JWT authorizer. For machine-to-machine `client_credentials`, bind it with `allowedClients` (the app-client id) — *not* `allowedAudience`, because those tokens carry no `aud` claim.
- 4. Outbound / two credential planes:** (1) gateway→Runtime via **SigV4** — a `GATEWAY_IAM_ROLE` provider whose service is `bedrock-agentcore`; and (2) server→Check Point management via credentials held *only* by the server. Keep plane-2 secrets in **AWS Secrets Manager**, fetched at container startup — never in the image, git, or the runtime's env map.
- 5. Private connectivity:** for VPC / on-prem management, use VPC Lattice private egress (managed VPC resources; self-managed + AWS RAM for cross-account). A PUBLIC Runtime's egress is dynamic and **rotates across a NAT pool**, so per-IP allowlisting on a source-restricted on-prem SMS won't hold — use a VPC + NAT Gateway with a **static Elastic IP**, or Smart-1 Cloud (public API, TLS verification stays on).

6. **Listing:** use DEFAULT mode (the stable Check Point toolset works with semantic search + cached `tools/list`); script `SynchronizeGatewayTargets` after any `@chkp` upgrade that changes tools.

#### FIELD-TESTED BUILD AVAILABLE

A complete **14-step AWS CLI build** of this architecture — Runtime container (ARM64, built on CodeBuild), Cognito inbound auth, gateway + `mcpServer` target, SigV4 outbound — was **run end to end**. It returned Check Point's full read-only tool catalog (**54 tools**; `tools/list` is cursor-paginated ~26/page — follow `nextCursor` for all of them) through a live AgentCore Gateway, and with real management credentials executed **live tool calls** (`management__init`, `show_hosts`) that returned actual objects from a Check Point management server. The field test happened to run in **AWS CloudShell**, but nothing here requires it: the same `aws bedrock-agentcore-control` calls run from a local terminal on **macOS, Windows (PowerShell/WSL), or Linux**, from the SDK, or declaratively via **CloudFormation / CDK / Terraform** — CloudShell is just one host. Step-by-step commands, the full deploy-path matrix, and the connectivity gotchas (rotating Runtime egress; on-prem reachability / asymmetric routing) are in the companion runbook (`README.md` — see *Choose how you deploy* and *Mode A*) and the one-shot `scripts/build.sh`.

#### THE IDEAL COMBINED TOPOLOGY

Check Point tools fronted by the gateway *and* Check Point AI Security policing that same gateway is the natural end state — but it is **not yet documented as a supported combined pattern**. Present it as a roadmap/open question, not a promise.

## Ways

# The Integration Ways — Choosing Your Mode

Check Point meets AgentCore across three families — **Tools** (the agent uses Check Point), **Enforcement** (Check Point polices the agent), and **Network/Edge** (Check Point secures the traffic). Statuses below are honest **as of 13 July 2026**; the full 21-way catalog + decision guide lives in the companion runbook. Headline ways:

# The Integration Landscape

Every way Check Point meets AWS Bedrock AgentCore — grouped by family, colour-coded by honest status (2026-07-13).

field-tested    GA (shipping)    announced-upcoming    inferred — GA parts, no vendor reference architecture

| TOOLS · agent <i>USES</i> Check Point | ENFORCEMENT · Check Point <i>POLICES</i> | NETWORK · secure the wire             |
|---------------------------------------|------------------------------------------|---------------------------------------|
| Mode A — @chkp as Gateway target      | Mode B — signals → AgentCore Policy      | Quantum FW R82.10 — MCP monitoring    |
| Direct MCP consumption (any client)   | Guard API (inline, per step)             | CloudGuard NS egress / SSRF control   |
| Portal remote OAuth MCP endpoint      | Embed Guard in agent code                | East-West inspection (Gateway → tool) |
| Multi-server aggregation              | Check Point WAF + Lakera (edge)          | Private connectivity (Lattice/PL/VPN) |
| Runtime-hosted @chkp (no gateway)     | Infinity GenAI Protect (workforce)       |                                       |
| Classic Bedrock Agent (ActionGroups)  | AI Red Teaming (pre-ship gate)           |                                       |
| Backend variants: SMS · S1C · SASE    | AI Defense Plane (AWS Marketplace)       |                                       |

Two deep-dives in the runbook: Mode A (field-tested build) and Mode B (guardrail). The full 21-row catalog with sources is one expand away.

Figure 3 — The integration landscape: three families — Tools (the agent *uses* Check Point), Enforcement (Check Point *polices* the agent), and Network (Check Point secures the *traffic*) — colour-coded by honest shipping status.

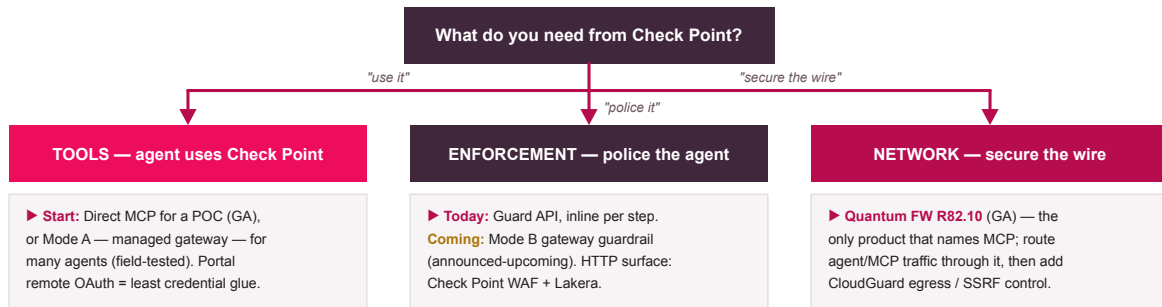
| Family      | Headline way                                                                  | Check Point                      | Status                          |
|-------------|-------------------------------------------------------------------------------|----------------------------------|---------------------------------|
| Tools       | <b>Mode A</b> — self-hosted <code>@chkp</code> MCP server as a Gateway target | @chkp MCP servers                | GA parts · <b>field-tested</b>  |
| Tools       | Direct MCP consumption (Strands / any client, no AgentCore)                   | @chkp MCP servers                | <b>GA (documented)</b>          |
| Tools       | Check Point Portal remote OAuth-hosted MCP endpoint as a 3LO target           | CP / Infinity Portal             | inferred (parts GA)             |
| Enforcement | <b>Mode B</b> — AI Security signals → AgentCore Policy (gateway guardrail)    | Check Point AI Agent Security    | <b>announced-upcoming</b>       |
| Enforcement | Guard API inline enforcement (per agent step)                                 | AI Agent Security (Lakera Guard) | <b>GA</b>                       |
| Enforcement | Check Point WAF + Lakera in front of GenAI/API/MCP endpoints                  | Check Point WAF                  | product GA · placement inferred |
| Network     | Quantum Firewall R82.10 — network-level MCP monitoring                        | Quantum Firewall R82.10          | <b>GA</b>                       |
| Network     | CloudGuard NS egress/SSRF control + private connectivity                      | CloudGuard Network Security      | inferred (parts GA)             |

**Which do I want?** To let the agent *use* Check Point → Tools (start with direct MCP or Mode A). To *police* the agent → Enforcement (optional — the guardrail is off unless you enable it; the AWS-native engine, AgentCore Policy / Bedrock Guardrails, is the default, or opt into Check Point's own AI Guardrail (Lakera Guard) with `--guardrail-provider lakera` for one guardrail story across AWS and Azure. Mode B is the announced marquee, plan-not-deploy; enforce today with either engine or the WAF). To secure the *wire* → Network (Quantum FW R82.10 is the only shipping product that names MCP).

#### WIRED IN THIS REPO — CHECK POINT AI GUARDRAIL (LAKERA GUARD)

The guardrail is **entirely optional** — the `@chkp` MCP servers run without it — and when it is enabled the customer **chooses the engine**. The **AWS-native engine stays the default provider** (AgentCore Policy / Bedrock Guardrails, the Mode B substrate; `--guardrail-provider gateway`), so teams already invested in the cloud's own guardrail keep it. Check Point's own **AI Guardrail (Lakera Guard)** is a **drop-in opt-in** — `chkpmcpaws chat --guardrail --guardrail-provider lakera` (or `CHKP_GUARDRAIL_PROVIDER=lakera`) — that screens the prompt with one `POST api.lakera.ai/v2/guard` *before* any model/tool call; a flagged prompt is blocked (set `LAKERA_API_KEY` / `LAKERA_PROJECT_ID`). It is **Check Point's own GA AI security** and is **identical on the Azure port** — one guardrail story across both clouds — so customers who want Check Point's unified-across-clouds engine choose Lakera. We do not force it.

## Which way do I want?



Rule of thumb: only Mode A, Direct MCP, Guard API, GenAI Protect, AI Red Teaming and Quantum FW R82.10 are shipping today; Mode B is announced, the rest are reference-architecture patterns.

Figure 4 — Choosing a mode: match the customer's goal (use / police / secure the wire) to a family and a headline way.

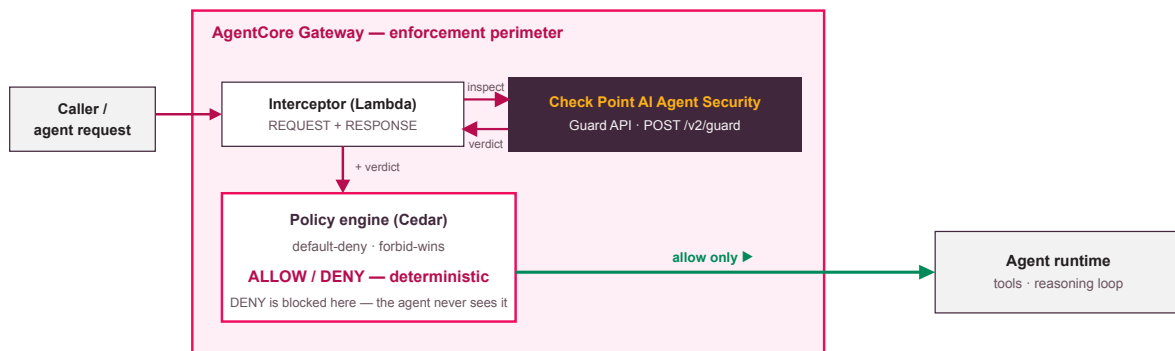
## MODE B AVAILABILITY — READ CAREFULLY

The Check Point-branded **Mode B** integration is **announced / upcoming** (13 Jul 2026) — no GA date, no public step-by-step. What *is* GA and buildable today is the AWS substrate it plugs into (**AgentCore Policy + Bedrock Guardrails-in-Policy**, Cedar allow/deny at the gateway) and Check Point's inline engine (**Guard API**, `api.lakera.ai/v2/guard`). Deploy deterministic enforcement now with AWS-native detectors; wire Check Point's *specific* signal in when the official mechanism ships.

## Mode B — Check Point AI Security as a gateway guardrail

Deterministic allow / deny at the AgentCore Gateway perimeter — outside the agent's reasoning loop.

announced-upcoming (2026-07-13)



The Cedar allow/deny sits **outside the agent's reasoning loop** — a prompt-injection inside the agent cannot override a gateway *forbid*.

Figure 5 — Mode B guardrail flow (**announced-upcoming**): the caller hits the gateway, an interceptor consults the Check Point / Guard API, and a Cedar policy returns a deterministic allow/deny *before* the Runtime is invoked — enforcement outside the agent's reasoning loop.

### DON'T OVER-CLAIM

AWS's own "AWS WAF for AgentCore Gateway" is not Check Point WAF. There is no "one-click" Check Point AgentCore target (Mode A is self-assembled; `@chkp` servers are stdio-first and mostly read-only). The **Bedrock-Guardrails baseline is AWS-native**, not Check Point. And the **AI Factory Security Blueprint**, the **AI Runtime Security Discovery Workshop**, and a **Cyata ↔ AgentCore Identity** binding are *not* AgentCore integrations — don't cite them as such.

09

## Summary & Honesty Guardrails

1. **The problem.** Connecting AI agents to real tools via MCP creates (a) configuration sprawl, (b) a new attack class where tool descriptions and results become injection vectors, and (c) credential/audit gaps because MCP auth is optional and HTTP-only. Real incidents already exist (postmark-mcp, Sept 2025).
2. **The pattern.** An MCP gateway is the control point that solves all three: one endpoint per client, centralized OAuth 2.1 with audience-bound tokens, tool allowlisting/hashing, human-in-the-loop, credential brokering, and unified audit.
3. **AWS AgentCore Gateway** is a mature, fully-managed example (GA Oct 2025) — a good backbone for AWS-centric customers.
4. **Check Point's two roles:** *tools* (the `@chkp` MCP servers query your estate in natural language, credentials stay server-side) and *enforcement* (CloudGuard WAF + Lakera dual-layer ML and Quantum Firewall R82.10's native MCP monitoring provide the runtime guardrail a gateway alone doesn't give you). The June 2026 announcement wires Check Point AI Security signals into AgentCore gateway policy.
5. **Honesty guardrails.** The Check-Point-tools-behind-AgentCore design is a reference architecture we assembled, not a shipped joint solution; the officially announced integration is the *guardrail* one. Recommend defense-in-depth: read-only least-privilege Check Point API keys, secrets in Secrets Manager, gateway auth via Cognito/OAuth, plus Check Point runtime guardrails in front of agentic usage.

### VERSION-SENSITIVITY NOTICE

This is a fast-moving area. As of `2025-11-25` : MCP spec baseline is `2025-11-25` (a `2026-07-28` release candidate exists but is not yet published); `@chkp/quantum-management-mcp` is at `1.x` (stable 1.x since 1.0.1, Nov 2025), supporting Management API v2.1 (R82.10+); AgentCore Gateway is GA (Oct 2025). `Node.js 20+` recommended for the Check Point servers; disable telemetry with `TELEMETRY_DISABLED=true`.

Prepared for customers who want to experiment with Check Point's open-source MCP servers. The "Check Point MCP server as an AgentCore target" design is a synthesized reference architecture, not an announced

joint AWS + Check Point solution. Verify all claims against current primary documentation.

## PRIMARY SOURCES

MCP specification — [modelcontextprotocol.io/specification/2025-11-25](https://modelcontextprotocol.io/specification/2025-11-25)

Check Point MCP servers — [github.com/CheckPointSW/mcp-servers](https://github.com/CheckPointSW/mcp-servers)

AWS Bedrock AgentCore Gateway — [docs.aws.amazon.com/bedrock-agentcore](https://docs.aws.amazon.com/bedrock-agentcore)

AgentCore MCP-server target — [gateway-target-MCPservers.html](https://gateway-target-MCPservers.html)

Check Point + AgentCore integration — [checkpoint.com](https://checkpoint.com) press release

OWASP MCP Security Cheat Sheet — [cheatsheetseries.owasp.org](https://cheatsheetseries.owasp.org)

Tool-poisoning research — [invariantlabs.ai](https://invariantlabs.ai)

postmark-mcp incident — [snyk.io](https://snyk.io)